

5 指派问题与匈牙利算法：LP 为什么自带整数解

第 2 章的涨价拍卖只讲了思路。这一章把它写成完整的算法：一次加一个任务，沿一条改派链调整；再弄清它凭什么成立，以及加上哪种约束它就会失效。

学完本章，你能

1. 说清新任务到达时最优派法怎么变：一条沿紧边的改派链，边际成本等于新任务的最终价格。
2. 手写 $O(n^3)$ 的匈牙利算法（Dijkstra 版），解释标签、前驱和价格更新各自的含义。
3. 证明指派约束矩阵全单模，并判断给它加一行之后整数性还在不在。
4. 处理车多任务少、任务多车少、禁行和延后罚金；用“阈值加匹配”解瓶颈指派，读懂 Hall 证书。

本章信息

用时	阅读与交互 80-110 分钟，Lab 2.5-3 小时
前置	第 1、2 章（指派模型、LP 松弛、价格、约化成本、紧边）。会读 numpy 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含一个手写算法，pytest 自动判分

PART 1
数学规划基础

PART 2
组合优化

PART 3
调度理论

PART 4
路径与波次

PART 5
大规模方法

PART 6
启发式

PART 7
多智能体时空规划

PART 8
动态与不确定

第二部分 组合优化：指派问题与匈牙利算法（本章）→ 最短路与网络流 → 带容量的指派与背包 → 旅行商与车辆路径入门。

5.1 任务陆续到达：一条改派链

第 2 章把整张价格表一次涨完。真实的派车窗口里，任务是陆续到达的：已经按最优派好了前 k 个任务，第 $k + 1$ 个到了，最优派法要怎么变？

本章沿用第 2 章的场景：6 辆车（V6 是备用车）、5 个任务，代价矩阵不变，最优总时间 124 s。任务按 T1 到 T5 的顺序到达。前四个到达时不需要想什么：每个任务拿走当时最便宜的空闲车。T1 拿 V3 (6 s)，T2 拿 V5 (23 s)，T3 拿 V2 (32 s)，T4 拿 V4 (9 s)。这四次到达，最优派法都没有动过已经派好的任务。

T5 到达时情况变了。它离 V5 最近 (10 s)，可 V5 正在做 T2。此刻空闲的车只有 V1 和 V6，去 T5 分别要 82 s 和 74 s。

先预测 T1-T4 已按最优派好 (T1←V3, T2←V5, T3←V2, T4←V4)。T5 到达，此刻空闲的车只有 V1 (82 s) 和 V6 (74 s)。总时间增量最小的做法是？

派最便宜的空闲车 V6，不动别人 (+74 s)

V5 改做 T5，T2 改派给空闲的 V6 (+62 s)

V5 改做 T5，T2 改派给 V2，T3 改派给 V1 (+54 s)

预测正确。 不改派要 74 s；让 T2 改给 V6 是 $10 + 75 - 23 = 62$ s；沿改派链 T5←V5、T2→V2、T3→V1 是 $10 + (61 - 23) + (38 - 32) = 54$ s。被改动的两个任务都变贵了，但 T5 省得更多。下面的图已经解锁，点“下一个到达”看这条链。

到达顺序

T1 → T5

T5 → T1

2 楼任务先到

随机顺序

2F

货位

5

4

3

2

1

主巷道

0

1

2

3

4

5

6

7

8

9

V4

T4

9s

T2

61s

T5

10s

V5

1F

货位

5

4

3

2

1

主巷道

0

1

2

3

4

5

6

7

8

9

提升机

V6

V1

V3

T3

38s

T1

6s

V2

■ 车（可拖动）

○ 任务（可拖动）

— 沿用的派法

— 这一步被改动的派法

- - 经提升机换层

已到达

5 / 5

刚到：T5

最优总时间

124 s

已到达任务上的最优

不改派的总时间

144 s

多 20 s

到达	任务	边际成本	不改派	改派链
1	T1	6 s	6 s	1 ●
2	T2	23 s	23 s	1 ●
3	T3	32 s	32 s	1 ●
4	T4	9 s	9 s	1 ●
5	T5	54 s	74 s	3 ●●●
合计		124 s	144 s	

T5 到达：改派链，共 3 步

1. **T5** ← V5 (10 s, 新增)
2. **T2** : V5 → V2 (23 → 61 s, **+38**)
3. **T3** : V2 → V1 (32 → 38 s, **+6**)

净增量 = 边际成本 = 54 s = 直接选最便宜的车 10 + 改派多付 44

默认场景：两层 · 6 车 · 5 任务（第 2 章） · 最优总时间 124 s

图 5-1 任务陆续到达。选一个到达顺序，一步步放任务进来。蓝线是沿用的派法，橙线是这一步被改动的，灰色虚线是被改动之前的。表里的“不改派”是每次都拿最便宜的空闲车。拖动车辆或任务可以换场景，也可以用 Tab 选中后按方向键移动。

读图

前四个任务，贪心和最优没有区别。每次到达，最便宜的空闲车正好就是最优的选择，改派链只有一个环节：新任务拿走一辆空闲车。

第五个任务，最优要动两个已经派好的任务。最优做法不是去找空闲的车，而是让 V5 转去做 T5，T2 改派给 V2（多 38 s），T3 再改派给 V1（多 6 s）。三个改动加起来的净增量是 54 s，比“不改派”的 74 s 少 20 s：被改派的 T2、T3 都变贵了（合计 +44 s），但 T5 从 74 s 降到 10 s，省下 64 s。

边际成本随顺序变，总和不变。默认顺序下五次到达的边际成本是 6、23、32、9、54，合计 124。换一个顺序，各步的边际成本会不同，最后的总和仍然是 124：每一步之后的派法，都是“当时已到达任务”上的最优。

贪心是长度为 1 的改派链

“拿最便宜的空闲车”只允许一种改动：新任务直接占一辆空闲车。匈牙利算法允许沿任意长的链改派，并且在所有链里挑最便宜的一条。下一节说这条链怎么找。

5.2 一次插入的内部：约化成本上的最短路

找最便宜的改派链，是一个最短路问题。边的长度是第 2 章的约化成本，路径从新任务出发，到任意一辆空闲车结束。

先把记号对齐。任务价格 u_j （可正可负），车价格 $v_i \geq 0$ ，约化成本 $r_{ij} = c_{ij} + v_i - u_j \geq 0$ ， $r = 0$ 的边是紧边。算法一直维持三件事：所有约化成本非负；已配对的边都是紧的；空闲的车价格为 0。第 2 章的证书检查器说的正是这三条，所以只要它们在，当前派法就是最优的。

新任务 j_0 到达。要给它找车，可以直接选一辆车 i_1 。如果 i_1 正被任务 j_1 占着， j_1 就得搬家，选一辆新车 i_2 ，依此类推，直到碰到一辆空闲车。这是一条交替路：任务 $\rightarrow$ 车 $\rightarrow$ 车原来的任务 $\rightarrow$ 车……每一步“任务选车”的代价是这条边的约化成本，“车推出原来的任务”不花代价（那条边是紧的）。所以：

边际成本 $u_0 + D$ $u_0 = \min_i(c_{i,j_0} + v_i)$ ，直接选最便宜车的成本； $D =$ 交替路上约化成本之和的最小值，是改派多付的部分

T5 的例子： $u_0 = 10$ （V5 的 10 s，此时 V5 的车价是 0）， $D = 44$ ，边际成本 54。“不改派”的做法是在空闲的车里挑，相当于只允许长度 1 的路径，多付 64 s。

求最短路用 Dijkstra。标签 d_i 是“从新任务出发，到达车 i 的最短距离”，等价于第 2 章里“要把新任务的价格再涨多少，车 i 才会变成紧边”。完整的一次插入分六步：

- 1 标价。 $u_{j_0} = \min_i(c_{i,j_0} + v_i)$ 。此后所有 $r_{i,j_0} \geq 0$ ，至少一个为 0。
- 2 初始标签。 $d_i = r_{i,j_0}$ ，前驱 $pred_i = j_0$ 。
- 3 扫描。在还没扫描的车里取 d 最小的 i^* （并列取编号小的）。标记它已扫描。
- 4 停止或展开。若 i^* 是空闲车，停止。否则设 j' 是它当前的任务： j' 也要搬家，对每辆未扫描的车 i ，若 $d_{i^*} + r_{i,j'} < d_i$ ，就更新 d_i 和 $pred_i = j'$ ，然后回到第 3 步。

- 5 **涨价**。记 $D = d_{i^*}$ 。每辆已扫描的车（终点除外） $v_i += D - d_i$ ，它当前的任务 u 涨同样多；新任务 $u_{j_0} += D$ 。
- 6 **翻转**。从 i^* 沿 $pred$ 回溯，任务逐个改派，得到改派链。

回放第几次到达 (任务按 T1 到 T5 顺序到达)

T1 到达

T2 到达

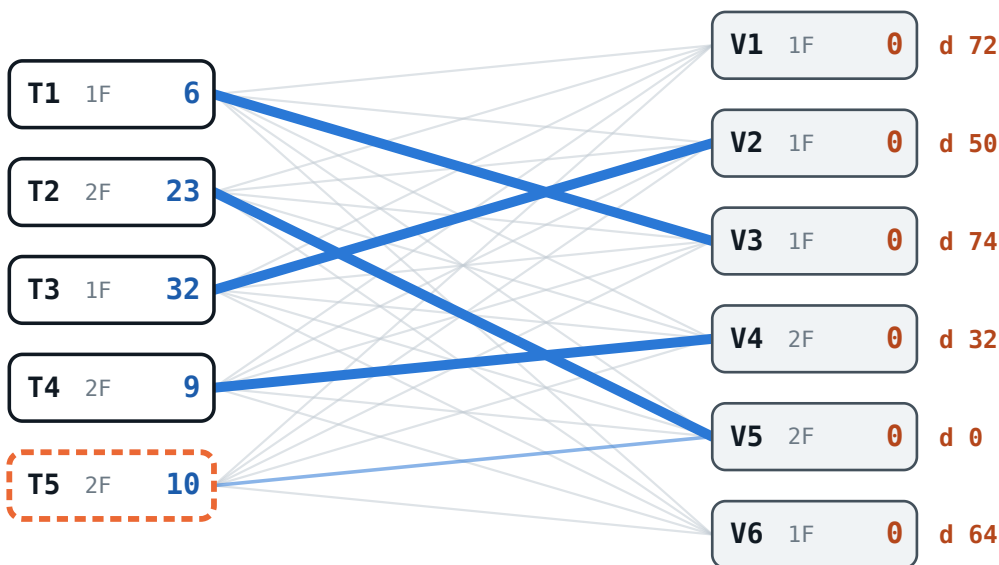
T3 到达

T4 到达

T5 到达

任务 u

车 v 标签 d



车	当前任务	车价 v	标签 d	前驱	状态
V1 (1F)	空闲	0	72	T5	待扫描
V2 (1F)	T3	0	50	T5	待扫描
V3 (1F)	T1	0	74	T5	待扫描
V4 (2F)	T4	0	32	T5	待扫描
V5 (2F)	T2	0	0	T5	待扫描
V6 (1F)	空闲	0	64	T5	待扫描

标签 d：新任务要涨多少价，才能让这辆车变成紧边。“前驱”是最短路上把这辆车“够到”的任务。

步 0 / 6

T5 到达。 标价 $u = \min(c + v) = 10$ (V5 : $c = 10$, 车价 $v = 0$) 。标签 d 是各车相对这个价格的约化成本：V1 72、V2 50、V3 74、V4 32、V5 0、V6 64。V5 的标签是 0，但它正被 T2 占着，直接拿走会让 T2 无车可用。

图 5-2 一次插入的回放 (默认数据的最终价格与 HiGHS 报告的顶点一致)。选 T5 看最长的一次：扫描 V5、V4、V2、V3，最后在 V1 停下，路径长度 $D = 44$ 。橙色边是最终的改派链，蓝色粗边是配对，虚线是已被扫描的“树”。

为什么这一步之后，价格仍然是公道的

涨价的规则看起来只是一组加减，要确认它没有把任何约化成本推成负数。三类边分开看：

- **配对的边（车和它的任务）**。车 v 涨了 $D - d$ ，任务 u 涨同样多， $r = c + v - u$ 不变，仍是 0。
- **路径上的新边**。Dijkstra 保证沿最短路每条边的“ d 差”恰好等于它的约化成本，涨价之后这条边的约化成本变成 0。改派翻转以后，它们成为新的配对，正好是紧的。
- **其他边（一端在树里，另一端不在）**。树里的任务涨了 $D - d$ （它的车被扫描时的标签），松弛保证 $d_i \leq d + r$ （ i 是未扫描的车， r 是这条边原来的约化成本），而 Dijkstra 又保证未扫描的车 $d_i \geq D$ ，所以 $r \geq D - d$ ：这条边只减少 $D - d$ ，不会变负。

终点车的增量是 $D - D = 0$ ，所以空闲的车价格仍是 0。三条不变量都保持住，第 2 章的互补松弛成立，当前派法对已到达的任务是最优的。同时可以看出：**新任务的最终价格 u_{j_0} 就是这次到达的边际成本**。其他配对的任务和车成对涨价，一个加一个减，对 $\sum u - \sum v$ 没有贡献，多出来的只有 $u_{j_0} = u_0 + D$ 。

和第 2 章的图 2-6 是同一件事

那里每一次“给被争抢的车涨价 δ ”，在这里是 Dijkstra 的一次扫描： δ 的累计就是标签 d 。第 2 章的涨价拍卖把许多次小涨价一步步做完；这里把它们合并成一次最短路，涨价一次到位。两者停在同一张价格表上，最终价格与 HiGHS 报告的一致。

5.3 写成代码： $O(n^3)$ 是怎么来的

上面六步写成 Python，大约三十行。Lab 练习 2 就是让你写这一段，并让它通过第 2 章的证书检查。

```

1 def insert(self, j0):                                     # 新任务 j0 到达
2     C, u, v, m = self.C, self.u, self.v, self.m
3     u[j0] = min(C[i][j0] + v[i] for i in range(m))      # 1 标价:最便宜的 成本 + 车价
4     d = [C[i][j0] + v[i] - u[j0] for i in range(m)]    # 2 标签 = 约化成本
5     pred = [j0] * m                                     # 前驱:谁“够到”了这辆车
6     scanned, order = [False] * m, []
7     while True:                                         # 3 Dijkstra:数组版, O(m) 取最
8         i_star = min((i for i in range(m) if not scanned[i]), key=lambda i: (d[i], i
9         scanned[i_star] = True
10        order.append((i_star, d[i_star]))
11        jp = self.task_of[i_star]
12        if jp is None:                                   # 4 碰到空闲车:停
13            break
14        for i in range(m):                               # 否则 jp 也搬家:松弛, O(m)
15            if not scanned[i]:
16                cand = d[i_star] + C[i][jp] + v[i] - u[jp]
17                if cand < d[i]:
18                    d[i], pred[i] = cand, jp
19        D = d[i_star]
20        for i, di in order[::-1]:                         # 5 涨价:扫描过的车和它们的任务一起
21            v[i] += D - di
22            u[self.task_of[i]] += D - di
23        u[j0] += D
24        i = i_star                                       # 6 沿前驱翻转:改派链
25        while True:
26            j = pred[i]
27            prev = self.vehicle_of[j]
28            self.vehicle_of[j], self.task_of[i] = i, j
29            if j == j0:
30                break
31            i = prev

```

复杂度

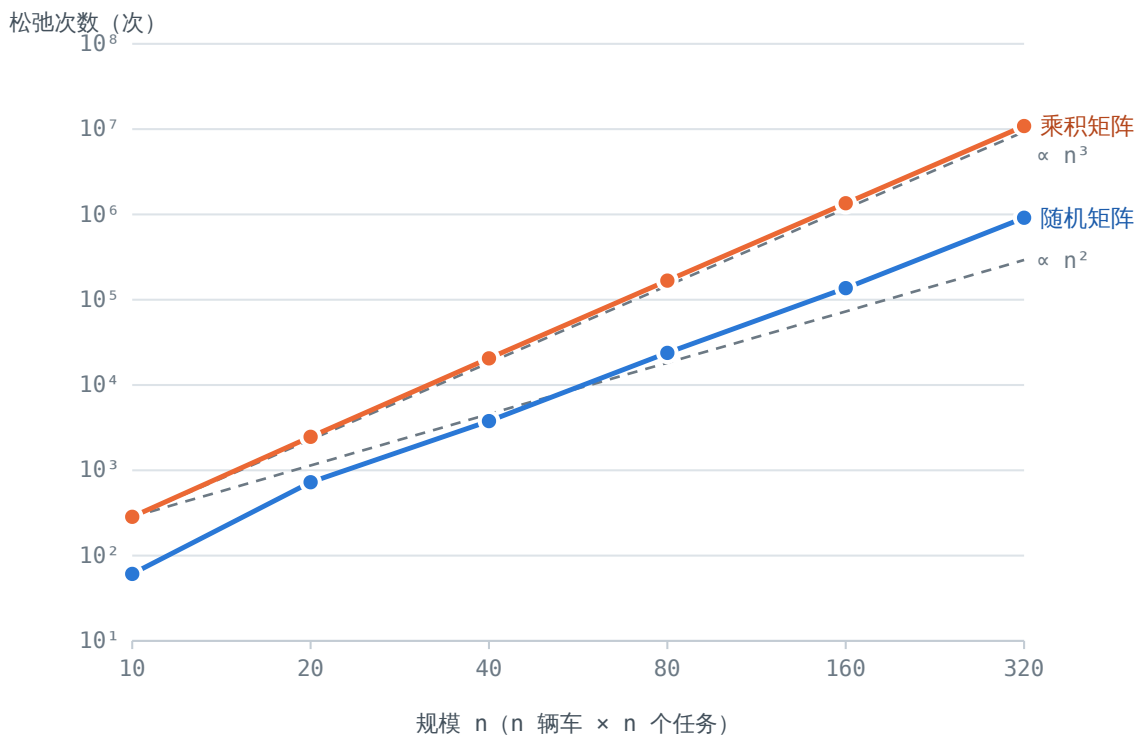
每辆车在一次插入里最多被扫描一次，所以第 3 步最多执行 m 次；每次扫描做一遍第 4 步的松弛，是 m 次比较。一次插入 $O(m^2)$ ， n 个任务共 $O(nm^2)$ ，方阵就是 $O(n^3)$ 。这里用数组而不是堆存标签：图是稠密的，每次扫描本来就要看全部 m 个标签，堆省不下什么。

$O(n^3)$ 是最坏情况，平时远没有这么糟。下面的实验数的是“扫描了多少辆车”和“松弛了多少次”，它们不依赖机器，每次跑都一样。

观察

松弛次数

扫描次数



乘积矩阵 $c_{ij} = i:j : n = 320$ 时扫描 51,360 次 ($= n(n+1)/2$)，松弛 10,871,520 次，松弛次数的拟合指数 ≈ 3.04 。

随机矩阵 (代价 1 到 10^6) : $n = 320$ 时扫描 4,272 次，平均每次插入 13.3 辆车，只占全部车的 4%，松弛次数的拟合指数 ≈ 2.71 。

在这组数据里，每次插入扫描的车数大致按 $\sqrt{n}$ 增长，所以总松弛次数的指数落在 2 和 3 之间，没有到最坏情形的 3。

随机矩阵每个规模是一个独立样本，曲线不会完全光滑。代价范围如果窄到 1 到 100，并列变多，扫描会明显增加。

点圆点可以看具体数字。

图 5-3 随机矩阵 (代价 1 到 10^6 的整数，几乎没有并列) 与乘积矩阵 $c_{ij} = i:j$ 上， $n \times n$ 指派的总扫描次数和松弛次数 (双对数坐标)。灰色虚线是 n^2 与 n^3 的参照斜率。乘积矩阵让每次插入都要扫描几乎全部的车，是这个算法的“最坏情形”。

随机矩阵上，每次插入只扫描一小部分车。价格把车分开了，新任务很快就碰到一辆空闲车， $n = 320$ 时平均每次十几辆，总松弛次数增长得比 n^3 慢。乘积矩阵上，第 k 次插入扫描 k 辆车，总数 $n(n+1)/2$ ，每次扫描又要松弛 n 次，三次方增长一目了然。Lab 练习 2 的测试里就有这一条。

和求解器比一比

SciPy 的 `linear_sum_assignment` 用的也是最短增广路，实现见 Crouse (2016)，再加上 C++ 的优化。下面是在生成本页数据的环境里的实测，规模是 $n \times n$ 、代价 1 到 10^6 的随机整数矩阵：

$n \times n$	手写 Python 增量版	<code>linear_sum_assignment</code>	LP (HiGHS 对偶单纯形)	MIP (HiGHS)
50 × 50	6.2 ms	0.02 ms	11.7 ms	35.9 ms
100 × 100	30.8 ms	0.23 ms	54.5 ms	325 ms
200 × 200	159 ms	1.1 ms	997 ms	8,103 ms
1000 × 1000	—	41.7 ms	—	—

手写 Python 版比 SciPy 慢两个数量级左右，差距来自解释器，不是算法。真正的差距在右边两列：把同一个指派问题当成 LP 或 MIP 交给 HiGHS，比 SciPy 慢两到四个数量级，规模越大差得越多。第 2 章的结论在这里再确认一遍：**识别出这是指派问题，比会调求解器重要得多。**

窗口重算，还是增量维护？

每个窗口有几百个任务的时候，从头解一次就是几毫秒，比维护增量状态更不容易出错，也更容易测试。增量的价值有两处：任务流非常密、单次响应必须在几十微秒内；以及改派链本身就是要给调度员看的解释，“T5 来了，V5 转去做它，T2 改给 V2，T3 改给 V1”。已经出发的车、已经开始的任务要先从问题里拿掉，只对还没执行的部分重算。

5.4 为什么 LP 自带整数解：全单模

第 1 章说指派的 LP 松弛顶点都是整数，给了两个理由：Birkhoff 的混合直觉，和一句克莱姆法则。这一节把它证明完整，再做一个能自动找反例的检查器，用来判断“加一行”之后还成不成立。

定义与定理

全单模

矩阵 A 全单模：它的每一个方子式（任取 k 行 k 列交叉出来的 $k \times k$ 子矩阵）的行列式都是 0、+1 或 -1。

1×1 的方子式就是元素本身，所以全单模矩阵的元素只能是 0 或 ± 1 。

Hoffman-Kruskal (1956)

若 A 全单模, b 是整数向量, 则 $\{x : Ax \leq b, x \geq 0\}$ 的每个顶点都是整数点。等式约束同理。

证明: 顶点是某个非奇异方程组 $Bx = b'$ 的唯一解。克莱姆法则说每个分量是两个行列式之比, 分母 $\det B = \pm 1$, 分子是整数, 所以分量是整数。

引理

二部图的顶点-边关联矩阵全单模。指派约束矩阵的每一列恰有两个 1, 一个在任务行, 一个在车行, 正是这种矩阵。

证明见下面, 只有四步。

引理对方子式的大小 k 归纳:

- 1 $k = 1$: 元素是 0 或 1。
- 2 $k > 1$, 任取一个 $k \times k$ 子矩阵 B 。若有一列全是 0, 行列式为 0。
- 3 若有一列恰好一个 1, 按这一列展开: $\det B = \pm$ 一个 $(k-1) \times (k-1)$ 子式, 由归纳假设是 0 或 ± 1 。
- 4 否则 B 的每一列恰有两个 1。原矩阵每列的两个 1 一个在任务行, 一个在车行, 所以 B 里也是这样: 把 B 里所有任务行相加, 等于把所有车行相加 (都是全 1 的向量)。行线性相关, $\det B = 0$ 。■

这个证明只用了一件事: 行可以分成两组, 每一列在两组里各有一个 1。这就是“二部”的含义, 也是后面所有反例失效的地方。

有了它, 第 1、2 章里几件没解释的事都有了出处:

- **LP 值 = 整数最优**。求解器在根节点就证明最优, 一次分支也不需要 (第 1 章的日志)。
- **价格也是整数**。 A 的转置同样全单模, 代价是整数时, 最优价格 u 、 v 可以取整数。这就是第 2 章和图 5-2 里价格、标签、约化成本全是整数的原因。
- **单纯形返回的顶点是真实派法**。最优派法不唯一时, 最优面上的中间点是分数解, 只有顶点才是整数, 所以第 1 章要求用对偶单纯形。

深入 Birkhoff-von Neumann: 混合直觉是它的特例

收起

行和、列和都是 1 的非负方阵 (双随机矩阵) 可以写成若干置换矩阵的加权平均。用 Hall 定理证明: 任意 k 行的支撑至少覆盖 k 列 (否则这 k 行的和 k 会挤进不到 k 列, 某列的和会超过 1), 所

以支撑里有一个完美匹配。取它，减去其中最小的元素，矩阵至少多一个零，行和列和变成同一个更小的常数，对它继续做。最多 n^2 轮结束。

这正是第 1 章挑战题“把分数解拆成真实派法”的算法，也是 5.5 节里 `decompose` 做的事。它和全单模是同一个事实的两种说法：顶点是整数，等价于任何可行点都是顶点的凸组合。

先预测 给 3 任务 × 3 车的指派约束再加一行。下面哪种加法会让矩阵不再全单模？

“V1、V2 合计至多接 1 个任务”

“V1→T1 与 V2→T2 不能同时出现”

两种都会

两种都不会

预测正确。 第一种把两辆车当成一个整体来限额，行仍然分成两组，还是全单模。第二种把两条互不相干的边绑在一起，检查器会找到行列式 ± 2 的子式。下面的检查器已经解锁。

在 3 任务 × 3 车的指派约束上加

不加 共享名额 重叠名额 冲突对 同车冲突 三边冲突 奇圈

	V1 T1	V1 T2	V1 T3	V2 T1	V2 T2	V2 T3	V3 T1	V3 T2	V3 T3
T1	1	.	.	1	.	.	1	.	.
T2	.	1	.	.	1	.	.	1	.
T3	.	.	1	.	.	1	.	.	1
V1	1	1	1	.	.	.	.	.	.
V2	.	.	.	1	1	1	.	.	.
V3	.	.	.	.	.	.	1	1	1

3 任务 × 3 车的指派约束。前 3 行：每个任务恰好一辆车；中间 3 行：每辆车至多一个任务。每一列是一个变量（车→任务），恰好有两个 1，一个在任务行，一个在车行。

方程式共 5,004 个，行列式取值 $\{-1, 0, 1\}$

det = -1 780

det = 0 3,213

det = +1 1,011

全单模。每一个方程式的行列式都在 $\{-1, 0, 1\}$ 里。配上任何整数右端项，LP 的顶点都是整数点，LP 值 = 整数最优。

图 5-4 全单模检查器。页面枚举这个矩阵的全部方程式，统计行列式的分布。只要出现一个行列式绝对值 ≥ 2 的方程式，矩阵就不是全单模，右边会显示找到的第一个反例（小的优先）以及它对应的分数解。橙色的行、列是反例所在的位置。

读图

基准。3×3 指派的 6 × 9 约束矩阵有 5,004 个方程式，行列式只取 0、+1、-1：3,213 个零，1,011 个 +1，780 个 -1。

共享名额仍是全单模。“V1 和 V2 合计至多 1 个任务”这一行，等于 V1 那一行加 V2 那一行，它把两辆车当成一个整体来限额。一般地，同一侧的车组只要“要么嵌套、要么不相交”（laminar 族），矩阵就仍是全单模：给每一行标上 $\pm$ 号，让同一列上被选中的车组行交替出现 -1、+1，再加上任务行的 +1，每列的和总落在 $\{0, \pm 1\}$ ，这就是 Ghouila-Houri 判据。

冲突对不是。“ $V1 \rightarrow T1$ 与 $V2 \rightarrow T2$ 不能同时出现”把两个互不相干的变量绑在一起，这一行不能并进任何一侧。页面找到的反例是一个 3×3 的子式，行列式 ± 2 ：解 $Bx = 1$ 得到 $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ 。“同车冲突”($V1 \rightarrow T1$ 与 $V1 \rightarrow T2$) 则相反，它被 $V1$ 那一行“至多一个任务”蕴含，没有新东西，矩阵仍是全单模。

两个重叠的名额也不是。 $V1 + V2 \leq 1$ 与 $V2 + V3 \leq 1$ 共用了 $V2$ ，行不能再分成两组。**奇圈**是另一种失效方式：三辆车两两之间都可以组队，关联矩阵是三角形的，行列式是 2。

不是全单模，不等于 LP 一定出分数解

全单模是“对任意整数右端项都保证整数顶点”的充分条件。具体的代价向量下，LP 的最优顶点可能碰巧仍是整数：在默认场景上加“ $V1 + V2 \leq 1$ 与 $V2 + V3 \leq 1$ ”，LP 值是 138 s，整数最优是 138 s。你失去的是保证，不是每一次的结果。所以要看的不是“这次碰巧整数”，而是“换一组代价还会不会”。

5.5 加一行，整数性就没了：冲突对与奇圈

全单模是个脆的性质。实际派车里最先破坏它的，往往不是复杂的约束，而是一句“这两个派法不能同时出现”。

冲突有很多来源：两条路线会在主巷道会车，两个任务共用一个月台，两辆车同时要占用提升机。第七部分的时空预约表会把它们检出来。在指派模型里，每一个冲突都是一行 $x_a + x_b \leq 1$ 。

默认场景里，最优派法是 $T1 \leftarrow V3$ 、 $T2 \leftarrow V2$ 、 $T3 \leftarrow V1$ 、 $T4 \leftarrow V4$ 、 $T5 \leftarrow V5$ ，总共 124 s。假设检测到 $V1 \rightarrow T3$ 与 $V2 \rightarrow T2$ 会冲突。这两条边都在最优派法里，所以它被排除了。用图 5-5 看 LP 怎么反应。

冲突对 (指派 + 一行)

奇圈组队 (一般图)

冲突的两条边

V1→T3 + V2→T2

V2→T2 + V5→T5

V3→T1 + V2→T2

	T1 1F	T2 2F	T3 1F	T4 2F	T5 2F
V1 1F	24	83	38 $x = \frac{1}{2}$	71	82
V2 1F	18	61 $x = \frac{1}{2}$	32 $x = \frac{1}{2}$	49	60
V3 1F	6	85	36	73	84
V4 2F	76	43	90	9	42
V5 2F	68	23	82	23	10
V6 1F	32	75 $x = \frac{1}{2}$	46	63	74

格子里是代价 c 。蓝底 = 没有冲突约束时的最优派法。点两个格子换一对冲突边（两条边的车和任务要互不相同）；LP 解里的分数会写在格子下方。

无冲突约束

124 s

第 2 章的最优

LP 值

128 s

分数解

整数最优

132 s

比 LP 多 4 s

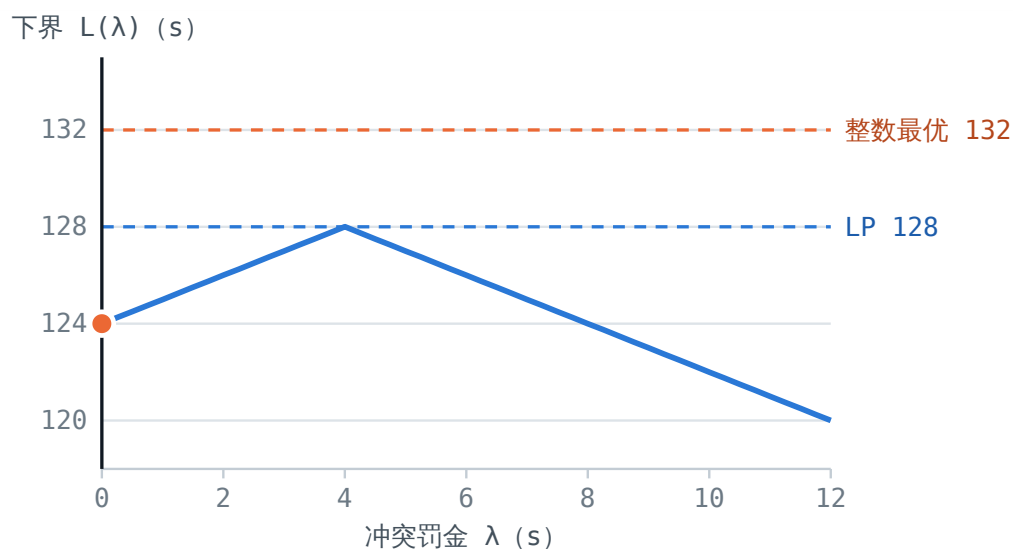
冲突：V1→T3 与 V2→T2 不能同时出现。LP 值 128 s，整数最优 132 s，差 4 s。LP 的解是几个真实派法的平均：

50% × T1←V3，T2←V2，T3←V1，T4←V4，T5←V5 · 124 s · 含 2 条冲突边

50% × T1←V3，T2←V6，T3←V2，T4←V4，T5←V5 · 132 s · 含 0 条冲突边

两条冲突边的 x 分别是 $1/2$ 、 $1/2$ ，和 $1 \leq 1$ ，约束满足。约束只管“平均”：一半时间做冲突的事，一半时间什么也不做。

冲突的影子价格 $\lambda^* = 4$ s：每禁止“这两个同时出现”，系统愿意多付约 4 s。



冲突罚金 $\lambda = 0$ s (两条边的代价各加 λ)

$L(0) = 124$ s，是这个带冲突约束的问题的下界。此时解出的指派含 2 条冲突边， L 的斜率是 +1 (还应加大 λ)。最高点 = LP 值 = 128 s，在 $\lambda^* = 4$ 处。

图 5-5 加一行以后的 LP。左：冲突对（默认场景，6 车 $\times$ 5 任务）。LP 用页面内置的单纯形法求解，整数最优用 720 种派法枚举，拉格朗日下界 $L(\lambda)$ 每个点解一次指派。右：三辆车两两组队，每辆车至多进一个组。

冲突对：LP 只管“平均”

默认的这一对，LP 是 128 s，整数最优是 132 s，差 4 s。LP 的解是两个真实派法的一半一半：一半是原来的最优派法（它同时含两条冲突边），一半是避开这两条边的最好派法（132 s）。两条冲突边的 x 各是 $\frac{1}{2}$ ，和是 $\frac{1}{2} + \frac{1}{2} = 1 \leq 1$ ，约束满足了。约束只限制“平均”，LP 就允许一半时间做冲突的事，另一半时间什么也不做。真实系统里没有“平均的窗口”，只能二选一，这 4 s 就是选择的价格。

只加了一行，为什么分数解只涉及两个派法？指派多面体加上一个半空间，新的顶点只能落在它的边上：两个相邻的顶点（两个真实派法，彼此只差一条交替路或一个环）之间的线段。这也解释了 Lab 里 `decompose` 每次只找到两个派法。

拉格朗日看这件事。把这一行乘以价格 $\lambda \geq 0$ 吸收进目标：两条冲突边的代价各加 λ ，再解一个普通指派，最后减去 λ ，得到 $L(\lambda)$ ，它是这个带约束问题的下界。拖动图里的滑块： $\lambda = 0$ 时是原来的 124； λ 涨到 4，两个候选派法（含两条边的 $124 + 2\lambda$ ，和避开它们的 132）扯平， L 到达最高点 128；再涨， L 又掉下去。最高点恰好等于 LP 值，最优的 $\lambda^* = 4$ s 是这个冲突的价格：每禁止“两条同时出现”一次，系统愿意为此多付大约 4 s。这是第 2 章“价格”的又一次出场，第五部分的拉格朗日松弛就是把这个做法用到大规模问题上。

奇圈：不是二部图

车队 (platoon) 把几辆车编成一队一起行驶，最简单的队是两两一组。任意两辆车之间都可以组队，所以这个配对问题的图**不是二部图**：三辆车 A、B、C 两两相连，构成一个三角形 (奇圈)。目标是让组队总收益最大，每辆车至多进一个组。三个收益都是 10 时，整数最优是 10 (只能成一组)，LP 却给出 15：三条边各取 $\frac{1}{2}$ ，每辆车的总量是 $\frac{1}{2} + \frac{1}{2} = 1$ ，约束满足了。

要修复它，靠的是问题自己的结构，换一个更聪明的求解器没有用。Edmonds 证明：对每个奇数大小的点集 S ，加上不等式“ S 内部的边至多取 $(|S| - 1)/2$ 条” (奇集割，也叫 blossom 不等式)，一般图匹配的 LP 就是整数的。这里 $S = \{A, B, C\}$ ，割是 $x_{AB} + x_{BC} + x_{AC} \leq 1$ 。勾选图里的复选框， $\frac{1}{2}$ 的解被切掉，LP 值降回整数最优。它是第 4 章“割平面”的一个结构化例子：割不是求解器随手找的，是从奇圈这个结构里读出来的。

三个收益不相等时，分数解也未必出现：当某一个收益比另外两个之和还大，LP 会直接选它。拖动滑块试试，出现分数解的条件是**每个收益都小于另外两个之和**，这时三条边各取 $\frac{1}{2}$ 的点 (总收益是三者之和的一半) 胜过任何单独一条边。

整数性什么时候还在，什么时候没了

还在：加的行是同一侧“嵌套或不相交”的车组名额；或者行本身被已有的行蕴含。**没了**：把两条不相干的边绑在一起 (冲突对)、名额相互重叠、引入车与车之间的配对 (奇圈)。没了以后，回到第 4 章的分支切割，或者用拉格朗日松弛把这一行“挂”在目标里，让剩下的部分仍是一个指派问题。第 29 章把任务分配和路径规划合到一起时，这些冲突约束会大量出现。

5.6 真实派车里的变形：车不够、走不通、可以等

教科书上的指派问题是方阵，每个人恰好做一件事。派车窗口里几乎没有方阵：车比任务多或少，有些车去不了某些任务，有些任务可以等下一个窗口。这些都不需要换算法，只需要把矩阵改成算法能吃的形状，并在事后检查它没有被“骗”。

车比任务多

什么都不用做。空闲的车价格为 0，匈牙利算法和 `linear_sum_assignment` 直接支持长方形。

任务可以等：补虚拟车

给每个任务加一辆虚拟车，它去这个任务的代价是延后罚金 p_j 。落在虚拟车上的任务就是被延后的任务。

用价格读：任务价格被封顶在 p_j 。被延后的任务价格恰好等于 p_j ，派出去的任务价格不超过 p_j 。

禁行：删边，或用足够大的 M

只能传矩阵的接口里，把禁行边的代价写成 $M = 1 + n \times \text{最大合法代价}$ 。任何用到禁行边的派法，一定比任何合法派法更贵。求解之后必须检查结果里有没有 M 。

检查到了，就是无解：返回 Hall 证书，不要返回一个含 M 的“最优解”。

延后罚金与虚拟车

禁行与无解

可用的车 (点击开关)

V1 (1F)

V2 (1F)

V3 (1F)

V4 (2F)

V5 (2F)

V6 (1F)

	T1 1F	T2 2F	T3 1F	T4 2F	T5 2F
V1 1F	24	83	38	71	82
V2 1F	18	61	32	49	60
V3 1F	6	85	36	73	84
V4 2F	76	43	90	9	42
V5 2F	68	23	82	23	10
V6 1F	32	75	46	63	74
虚拟车 ×5	60	60	60	60	60

灰色的行是没有开的车。最下面一行是虚拟车：每个任务一辆，去这个任务的代价是延后罚金 p 。蓝底是当前的最优派法。

延后罚金 $p = 60$ s (每个任务相同)

派出去

4 个

6 辆车可用

延后

1 个

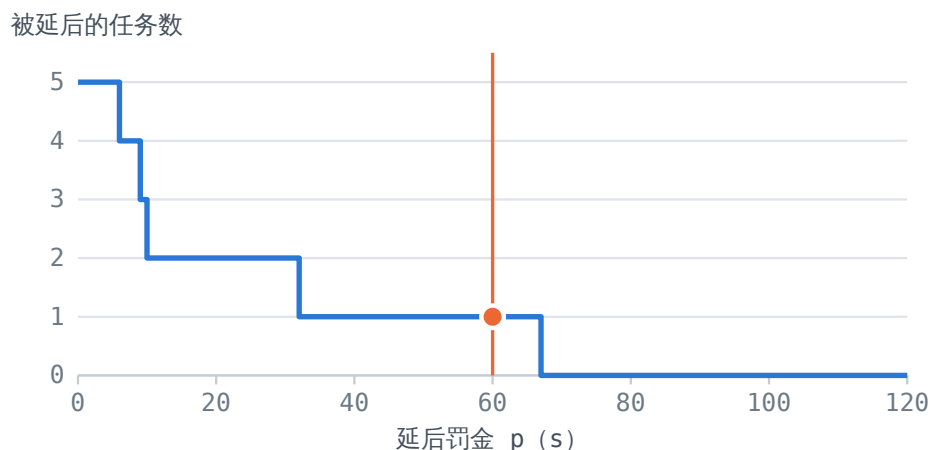
每个付 60 s

总时间

117 s

行驶 57 + 罚金 60

任务	结果	成本	价格 u
T1	V3	6 s	6
T2	延后	60 s	60 (= p)
T3	V2	32 s	32
T4	V4	9 s	26
T5	V5	10 s	47



罚金从 0 往上调： p 越过某个任务的边际成本，它就从“延后”变成“派出去”。被延后的任务数在 $p = 6、9、10、32、67$ s 处依次减少。

拖动上面的滑块，看竖线走到哪一级台阶。关掉几辆车，台阶会整体右移：车少了，每个任务的边际成本更高，要更高的罚金才值得派出去。

图 5-6 变形工坊。左：车比任务少时补虚拟车，罚金 p 决定谁被延后（折线是被延后的任务数）。右：禁行边与无解，“正确做法”是 $M = 1 + n \times \text{最大合法代价}$ 加事后检查，“常见写法”是直接写 $1e6$ ，不检查。

读图

罚金是任务价格的天花板。把 p 从 0 慢慢调高： p 很低时所有任务都被延后，总时间只有罚金； p 每越过一个任务的边际成本，就多派出去一个。表里“价格”一列，派出去的任务价格 $\leq p$ ，被延后的任务价格恰好 $= p$ 。这是第 2 章对偶的直接读法：虚拟车的公道条件 $u_j \leq p_j$ 。所以罚金不是一个随手写的大数，它是“这个任务最多值得为它付多少”，取值要有业务含义：超过它就宁愿等。

无解要被报告，不能被吞掉。选“不许换层（升降机停用）”，5 个任务里有 3 个在 2 楼，2 楼只有 V4 和 V5 两辆车，谁也救不了：**{T2, T4, T5} 只能用 {V4, V5}，3 个任务抢 2 辆车。**这一对集合就是无解的证书，检查它只需要看几个格子。“常见写法”不会发现这件事：它把禁行边写成 $1e6$ ，仍然返回一个“最优解”，总时间是一个含 $1e6$ 的数，用到了禁行的边。这种错误要靠事后检查才抓得住，Lab 练习 5 会给你 5 个带这类问题的“AI 写的”函数。

M 取多大

太小会出问题： M 只要小于“绕开这条禁行边要额外付的代价”，最优解就会去用它，而合法解其实存在。（Lab 里的 AI-B 就把 M 写成矩阵的最大值。）太大也有代价： M 过大，浮点数里合法代价的差别会被淹没。 $M = 1 + n \times \text{最大合法代价}$ 就够了：它比任何合法派法的总代价都大，又不会大到淹没数值。第 3 章 Big-M 的道理在这里原样成立： **M 不是随便写的大数，它有一个下限，也不该无限大。**

5.7 换个目标：最晚到达、Hall 证书与折中

第 1 章用一个辅助变量把“最晚到达最短”写成 MIP，LP 松弛只有 36.78 s，整数最优是 49 s，gap 近 25%。当时留了一句话：这个问题本身并不难，难的是那种写法。现在可以把它讲完整。

做法叫**阈值加匹配**。选一个阈值 τ ，只保留 $c_{ij} \leq \tau$ 的（车，任务）对，问：能不能让每个任务都配到一辆不同的车？这是一个二部图匹配问题，用的还是 5.2 节的增广路。 τ 越大边越多，“能配满”是单调的，所以可以二分，最小的可行 τ 就是最晚到达的最优值。

τ 太小的时候配不满。这里的“不行”不是“没找到”，可以拿出一个证据：

Hall 定理 (1935)

每个任务都能配上不同的车，当且仅当对任意一组任务 S ，它们的可用车的集合 $N(S)$ 至少和 S 一样大： $|N(S)| \geq |S|$ 。

配不满时，一定存在某组 S 有 $|N(S)| < |S|$ ，这就是证书：谁在抢谁，一眼可查。5.2 节的交替树从没配上的任务出发，长出来的就是这个 S 。

更进一步，把“最晚到达”当成一个预算，就能问：为了让最晚的车再早一点，总时间要多付多少？对每个 τ ，只保留 $c \leq \tau$ 的边，用匈牙利算法求最小总时间。扫描 τ ，得到（最晚到达，总时间）的全部折中，叫帕累托阶梯。第 1 章那个“最晚到达最短”的方案（总时间 146 s），只是阶梯最左边的一点。

先预测 总时间最优的派法（124 s）里，最晚到达是 61 s。把最晚到达的上限从 61 s 收紧一秒，到 60 s，总时间最少要多花多少？

约 1 s

约 4 s

12 s

22 s

预测正确。 V2 是唯一适合换层的车。上限 60 s 让它不能再去 T2 (61 s)：T2 要让给 V5，V5 原来的 T5 转给 V2 (60 s)，总时间 136 s，多 12 s。再压到 49 s 还要多 10 s，一共 22 s。下面的图已经解锁。

最晚到达上限 τ $\tau = 61 \text{ s}$

	T1 1F	T2 2F	T3 1F	T4 2F	T5 2F
V1 1F	24	83	38	71	82
V2 1F	18	61	32	49	60
V3 1F	6	85	36	73	84
V4 2F	76	43	90	9	42
V5 2F	68	23	82	23	10
V6 1F	32	75	46	63	74

格子里是代价。灰色 = 超过 τ ，不允许；蓝底 = 在允许的边里总时间最小的派法。配不满时，橙色框出违反 Hall 条件的那组任务和它们仅有的车。

状态

配得满

每个任务都有车

最小总时间

124 s

总时间最优

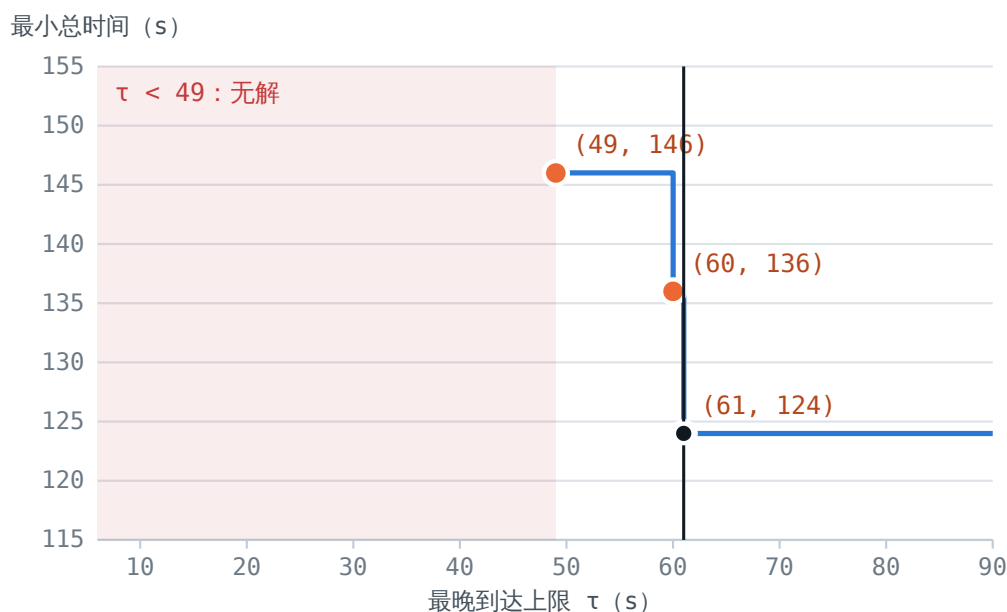
实际最晚到达

61 s

恰好被 τ 卡住

$\tau = 61 \text{ s}$: **配得满**。在只保留 $c \leq 61$ 的边里，总时间最小的派法是 $T1 \leftarrow V3$ ， $T2 \leftarrow V2$ ， $T3 \leftarrow V1$ ， $T4 \leftarrow V4$ ， $T5 \leftarrow V5$ ，总时间 **124 s**，最晚到达 **61 s**（橙框）。

这一点在帕累托阶梯上：想再压低最晚到达，总时间就必须付出更多。



最晚到达	总时间	比最优多	派法
49 s	146	+22	1←3 2←4 3←1 4←2 5←5
60 s	136	+12	1←3 2←5 3←1 4←4 5←2
61 s	124	0	1←3 2←2 3←1 4←4 5←5

图 5-7 阈值加匹配。拖动 τ ：配得满时显示允许的边里总时间最小的派法；配不满时给出 Hall 证书。下面的阶梯是每个 τ 对应的最小总时间，橙点是帕累托最优的三个折中。

读图

49 s 之前都不可行，证据是三个任务抢两辆车。把 τ 拖到 49 的前一档 (46 s)：T2、T4、T5 都在 2 楼，能在 46 s 内够到它们的只有 V4 和 V5。 τ 到 49 s，V2 到 T4 的 49 s 进入允许集合，Hall 条件恢复，配满。这也就证明了：默认场景里“最晚到达”不可能小于 49 s。（滑块只在矩阵里出现过的代价值上停，因为只有那些点上，允许的边集才会变。）

阶梯只有三级。(61 s, 124)：总时间最优，V2 坐提升机去 T2 要 61 s。(60 s, 136)：只压 1 秒，V2 不能再去 T2，T2 要让给 V5，V5 原来的 T5 转给 V2 (60 s)，总时间多 12 s。(49 s, 146)：再压 11 秒，代价是又多 10 s。**最晚到达每压一秒并不是等价交换。**把这条阶梯拿给调度员，比只给一个“最优解”有用：他可以指着阶梯选“最晚的车不许超过 60 秒”。

没有 gap。第 1 章的 MIP 写法在根节点只有 36.78 s，要靠分支和割平面补齐；阈值加匹配是精确的多项式算法，每个 τ 只是一次匹配和一次匈牙利。换写法，比换求解器管用。（这套做法叫 ϵ -约束法：把一个目标变成约束，另一个目标继续优化。它能求出帕累托阶梯上的每个点，而加权求和只能找到阶梯凸出去的那些点。）

5.8 小结与 Lab

- 1 新任务到达，最优派法沿一条交替路改派：新任务占一辆车，被占的任务再搬家，直到碰到一辆空闲车。这条链在约化成本上用 Dijkstra 找最短的，边际成本 = 新任务的最终价格 = $u_0 + D$ 。贪心只允许长度为 1 的链。
- 2 一次插入最多扫描 m 辆车，每次扫描 $O(m)$ ，共 $O(n m^2)$ ，方阵就是 $O(n^3)$ 。随机矩阵上增长得慢一些（本页实验里松弛次数的指数约 2.7），乘积矩阵是最坏情形。这是 SciPy `linear_sum_assignment` 的同类算法，比把它当 LP 或 MIP 交给 HiGHS 快两个数量级以上。
- 3 指派约束矩阵是二部图的关联矩阵，全单模：每列的两个 1 让任务行之和等于车行之和，所以任何方子式的行列式只能是 0、 ± 1 。于是 LP 值 = 整数最优，最优价格也可以取整数。
- 4 加一行会不会破坏整数性，看它能不能保持“行分成两组、每列各一个 1”的结构。“嵌套或不相交”的车组名额可以，冲突对、重叠名额、车与车的配对（奇圈）不行。失效时 LP 是几个真实派法的“平均”，只限制平均。
- 5 冲突有价格：拉格朗日下界 $L(\lambda)$ 的最优 λ^* 就是冲突的影子价格，最高点等于 LP 值。奇圈用奇集割修复，割是从问题结构里读出来的。
- 6 变形靠改矩阵，不换算法。虚拟车加延后罚金，价格被罚金封顶；禁行用 $M = 1 + n \times \text{最大合法代价}$ 并事后检查；无解时返回 Hall 证书，不返回含 M 的“最优解”。
- 7 “最晚到达最短”是阈值加匹配：多项式、精确、没有 gap，无解时有 Hall 证书。扫描阈值得到（最晚到达，总时间）的帕累托阶梯，默认场景只有 (61, 124)、(60, 136)、(49, 146) 三级。

这一章在你的课题里的位置

四向车与提升机联合调度的派车窗口，核心就是这一章。一个稳妥的实现是：任务流用增量插入，窗口边界用整窗重算兜底，每次都把改派链作为解释输出（“T5 到了，V5 转去做它，T2 改给 V2，T3 改给 V1”）；上线前用第 2 章的证书检查器加本章 Lab 的审查思路做验证。

两类约束会把你带出全单模的范围，都在后面的章节里回来。路线冲突来自时空预约表（ReservationManager 的原型在第 26 章），它们进入指派模型就是本章的冲突对，冲突的价格

会一直陪你到第 29 章的任务分配与路径联合规划。车队 (PlatoonManager 的原型在第 28 章) 里的两两组队是一般图匹配，奇圈和奇集割是它的基本工具。

术语表

中文	English	在本章的含义
增广路 / 改派链	augmenting path	从新任务出发的交替路：任务占车，车原来的任务搬家……直到一辆空闲车
边际成本	marginal cost	新任务到达后最优总时间的增量，等于新任务的最终价格
标签、前驱	label, predecessor	Dijkstra 里到达每辆车的最短距离，以及最短路上的上一个任务
约化成本、紧边	reduced cost, tight edge	$c + v - u$ ；为 0 的边
全单模	totally unimodular	所有方子式的行列式是 0、 ± 1 的矩阵
Hoffman-Kruskal 定理	—	全单模矩阵 + 整数右端 $\Rightarrow$ 多面体的顶点全是整数点
Birkhoff 多面体	Birkhoff polytope	双随机矩阵的集合，顶点是置换矩阵
冲突对	conflict pair	$x_a + x_b \leq 1$ ，两个派法不能同时出现
奇集割	odd-set (blossom) inequality	奇数大小的点集内部至多取 $(S -1)/2$ 条边
拉格朗日下界	Lagrangian bound	把一行乘以价格 λ 吸收进目标后得到的下界 $L(\lambda)$
延后罚金、虚拟车	deferral penalty, dummy vehicle	让任务可以不派车的补法；价格被罚金封顶
瓶颈指派	bottleneck assignment	让最大的那个代价最小
Hall 证书	Hall violator	一组任务 S ，它们能用的车 $N(S)$ 比 S 还少
帕累托阶梯	Pareto front	(最晚到达，总时间) 里没有被支配的全部折中

对照教材

- Wolsey, *Integer Programming* (第 2 版, Wiley 2020)：第 3 章 Well-Solved Problems (全单模、Hoffman-Kruskal)、第 4 章 Matchings and Assignments (匹配、匈牙利算

法、奇集割)。

- Burkard、Dell’Amico、Martello , *Assignment Problems* (SIAM , 修订版 2012) : 指派问题的专著, 含算法、变形和瓶颈指派。
- Ahuja、Magnanti、Orlin , *Network Flows* (Prentice Hall , 1993) : 第 12 章 Assignments and Matchings。
- Bertsimas & Tsitsiklis , *Introduction to Linear Optimization* : §7.8 指派问题与拍卖算法。
- Kuhn , “The Hungarian method for the assignment problem” (1955) ; Munkres , “Algorithms for the assignment and transportation problems” (1957) 。 Edmonds & Karp (1972) 与 Tomizawa (1971) 给出了带势的最短增广路。Jonker & Volgenant , “A shortest augmenting path algorithm for dense and sparse linear assignment problems” (Computing , 1987) 。
- Crouse , “On implementing 2D rectangular assignment algorithms” (IEEE Trans. Aerospace and Electronic Systems , 2016) : SciPy [linear_sum_assignment](#) 的实现依据。
- Hoffman & Kruskal , “Integral boundary points of convex polyhedra” (1956) ; Birkhoff (1946) 与 von Neumann (1953) ; Edmonds , “Maximum matching and a polyhedron with 0,1-vertices” (1965) ; Hall , “On representatives of subsets” (1935) 。
- Gabow & Tarjan , “Algorithms for two bottleneck optimization problems” (J. Algorithms , 1988) : 瓶颈指派的多项式算法。

本章 Lab

代码包 `ch05-assignment-lab.zip` 随本章一起发放。依赖是 `numpy`、`scipy`、`pytest`；判分用暴力枚举得到的最优值、Hall 证书的独立检查和一批随机场景，不依赖你的代码，也不依赖求解器。

练习	内容	对应
ex1	全单模检查器：枚举方子式、找反例，判断 6 种“加一行”	5.4
ex2	手写增量匈牙利： <code>insert(j)</code> 返回扫描过程、改派链、边际成本，通过证书检查与“黄金轨迹”	5.1-5.3
ex3	长方形、禁行、延后罚金的包装，无解时给出 Hall 证书	5.6
ex4	瓶颈指派、字典序最优、帕累托阶梯	5.7
ex5	审 5 个 AI 写的派车函数：不看源码，只靠运行结果归类问题	5.6
挑战	冲突对的拉格朗日下界：在 λ 上二分，与 <code>linprog</code> 对拍	5.5

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex2.py -q # 单个练习

# 确认判分本身没问题：参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行；跑完测试后，终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 06

最短路与网络流

这一章的算法里，Dijkstra 加一组价格（势）已经露面了：每次找一条最短路，沿它调整，再更新价格。这就是最小费用流的“逐次最短路”。下一章把指派放回网络里：最短路、最大流、最小费用流，以及为什么网络矩阵天然不需要分支。

5.9 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 已经按最优派好前 k 个任务，第 $k + 1$ 个到达。得到新的最优派法，最直接的做法是？

把新任务派给最便宜的空闲车，不动别人

从新任务出发，在约化成本上找一条通往空闲车的最短交替路，沿路改派

把 $k + 1$ 个任务重新写成 MIP 解一遍

把新任务排到队尾，下一个窗口再说

正确。 最短交替路就是最便宜的改派链，用 Dijkstra 在约化成本上找。“不改派”只是长度为 1 的链，可能比最优多付 20 s（默认场景的 T5）。从头重算也能得到同一个答案，但这一题问的是最直接的做法。

Q2 T5 的边际成本是 $54 = 10 + 44$ 。这两个数分别是什么？

10 是直接选最便宜的车的成本 u_0 ，44 是改派链多付的部分 D

10 是链的长度，44 是价格

10 是 V5 的车价，44 是 T5 的价格

没有含义，只是碰巧加起来

正确。 边际成本 = $u_0 + D$ 。 $u_0 = \min(c + v)$ 是“直接拿最便宜的车”的成本， D 是交替路上约化成本之和，是为了不让被占用的任务无车可用而多付的部分。它也等于新任务的最终价格。

Q3 涨价之后，已经配对的边为什么仍是紧边？

配对的车和任务同时涨了同样多，约化成本 $c + v - u$ 不变

配对的边不参与涨价

涨价量取 0

最后会重新配对一遍

正确。 已扫描的车涨 $D - d$ ，它当前的任务同涨 $D - d$ 。约化成本 $c + v - u$ 里 v 和 u 一个加一个减，差不变，仍是 0。

Q4 指派约束矩阵的方程式，行列式为什么只能是 0、+1、-1？关键的一步是？

矩阵的元素都是 0 或 1

若子矩阵每列都恰有两个 1（一个在任务行、一个在车行），任务行之和等于车行之和，行线性相关，行列式为 0；否则按只有一个 1 的列展开，用归纳

任何方阵都可逆

求解器返回的解总是整数

正确。 只有 0、1 的元素不足以保证，二部结构才是关键：行分成两组，每列在两组里各有一个 1。冲突对、奇圈都破坏了这一点。

Q5 给指派约束加下面哪一行，会破坏全单模？

V1、V2 合计至多接 1 个任务

V1→T1 与 V1→T2 不能同时出现（同一辆车）

V1→T1 与 V2→T2 不能同时出现

V3 至多接 1 个任务（已有的行）

正确。 只有第三个把两条互不相干的边绑在一起。第一个是嵌套或不相交的车组名额，第二个被“V1 至多一个任务”蕴含，第四个本来就在。

Q6 三辆车两两组队（每辆车至多一个组），三个收益都是 10。LP 值和整数最优分别是多少？怎么修复？

LP 15，整数 10；加奇集割 $x_{AB} + x_{BC} + x_{AC} \leq 1$

LP 10，整数 15；再加变量

都是 15，不用修复

LP 30，整数 10；换整数求解器

正确。 三条边各取 $\frac{1}{2}$ ，每辆车的总量是 1，总收益 15；真实组队只能选一组，收益 10。三个点里至多配成 $\lfloor 3/2 \rfloor = 1$ 对，奇集割把 $\frac{1}{2}$ 的点切掉。

Q7 禁行边用 $c = 1e6$ 表示，而这个窗口其实无解。这段代码会怎样？

抛出异常

仍然返回一个“最优解”，总时间含 $1e6$ ，派法用到了禁行边；必须事后检查，并返回 Hall 证书

自动返回空派法

返回不含禁行边的次优解

正确。 求解器只看矩阵，不知道 $1e6$ 是“禁止”。有解时两种写法一样，无解时它照样给出一个含 M 的“最优解”。要检查结果里有没有用到 M ，无解就返回 Hall 证书。

Q8 “最晚到达最短”为什么不必写成 MIP？

因为 MIP 求不出来

二分阈值 τ ，只保留 $c \leq \tau$ 的边，判断能否配满；最小的可行 τ 就是答案。配不满时有 Hall 证书说明谁在抢谁

因为总时间才是唯一合理的目标

因为必须用启发式

正确。 阈值加匹配是精确的多项式算法，没有 gap。扫描 τ ，每个 τ 再用一次匈牙利求最小总时间，就得到（最晚到达，总时间）的整条帕累托阶梯。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的增量匈牙利、单纯形法和枚举实时计算，并与 SciPy、HiGHS 和暴力枚举逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0）：增量算法在随机长方形矩阵上每个前缀都与 `linear_sum_assignment` 一致，全单模计数、LP/整数最优、帕累托阶梯与 Lab 的判分数据同源。耗时参数是教学用的整数近似，实测耗时来自生成本页数据的环境。